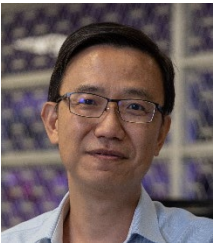


SC4172

Internet of Things - TinyML

Intro to Machine Learning (Updated 10 Jan 2026)



Oh Hong Lye
Senior Lecturer
CCDS, Nanyang Technological University.
Email: hloh@ntu.edu.sg



What is TinyML?

- Deployment of Machine Learning models on 'tiny' devices
 - Microcontrollers with **KByte to low MByte** range of memory.
 - Usually with **Bare metal system** i.e. no operating system support.
 - **Very low power consumption** typically in mW range.
- This course is based on
 - **TensorFlow Lite** for Microcontrollers.
 - **ARM Cortex-M** family.
- In this course
 - **Arduino Nano 33 BLE Sense** board.
 - **Adafruit Feather nRF52840 Sense** board.
 - **Cortex M4** microcontroller (64Mhz) with multiple sensors
 - **256kB** on-chip **RAM**, **1MB** on-chip **Flash**.



More on TinyML

- Typical **Neural Network (NN)** based model
 - Designed to operate in **high performance processor/GPU** system with huge memory.
 - Deals with complex events that requires complex NN architecture and require huge data set for training.
- **TinyML**
 - Target **simpler events** so uses a **simpler NN** architecture with lesser parameters.
 - **Only the inference stage** is implemented on the microcontrollers. Training of the NN model is still done on powerful machines to speed up the process.
- **Same methodology** can be applied to **edge computing** e.g. handphones. Which has higher performance but still one notch down wrt higher performance GPUs or neural processors.

Micro Controllers used in TinyML

- Low to mid-level performance (ARM Cortex-M series or lower)
- Small RAM and Flash memory footprint (Kbytes to low Mbytes)
- Computation confined to local system, no need for cloud connectivity. This result in
 - lower power consumption,
 - shorter latency and
 - enhanced data privacy.
- Low power consumption profile in mW range. Single battery with capacity in 10s to 1000mAh range, could last for years.
- Possible to apply energy harvesting (solar, vibration, heat, mechanical etc) as well.
- Can be deployed in remote area without much/any maintenance.

What does the study of TinyML involves?

- **Embedded system** development – hardware and software
- Understand the **characteristics of sensors** that transduce real life signals to digital domain
- Understand **Machine Learning** architectures
- Learn the relevant **techniques** and **tools** used in development, debug and profiling of the above tasks.
- **Sensor domains** may include the following
 - Audio
 - Vision (image and video)
 - Motion (Accelerometer, Gyro and Magnetometer)
 - Chemical
- An extension of the IoT (Internet-of-Things) domain to incorporate AI ability
 - $\text{IoT} + \text{AI} = \text{AIoT}$

Some (Possible) Use-case Examples

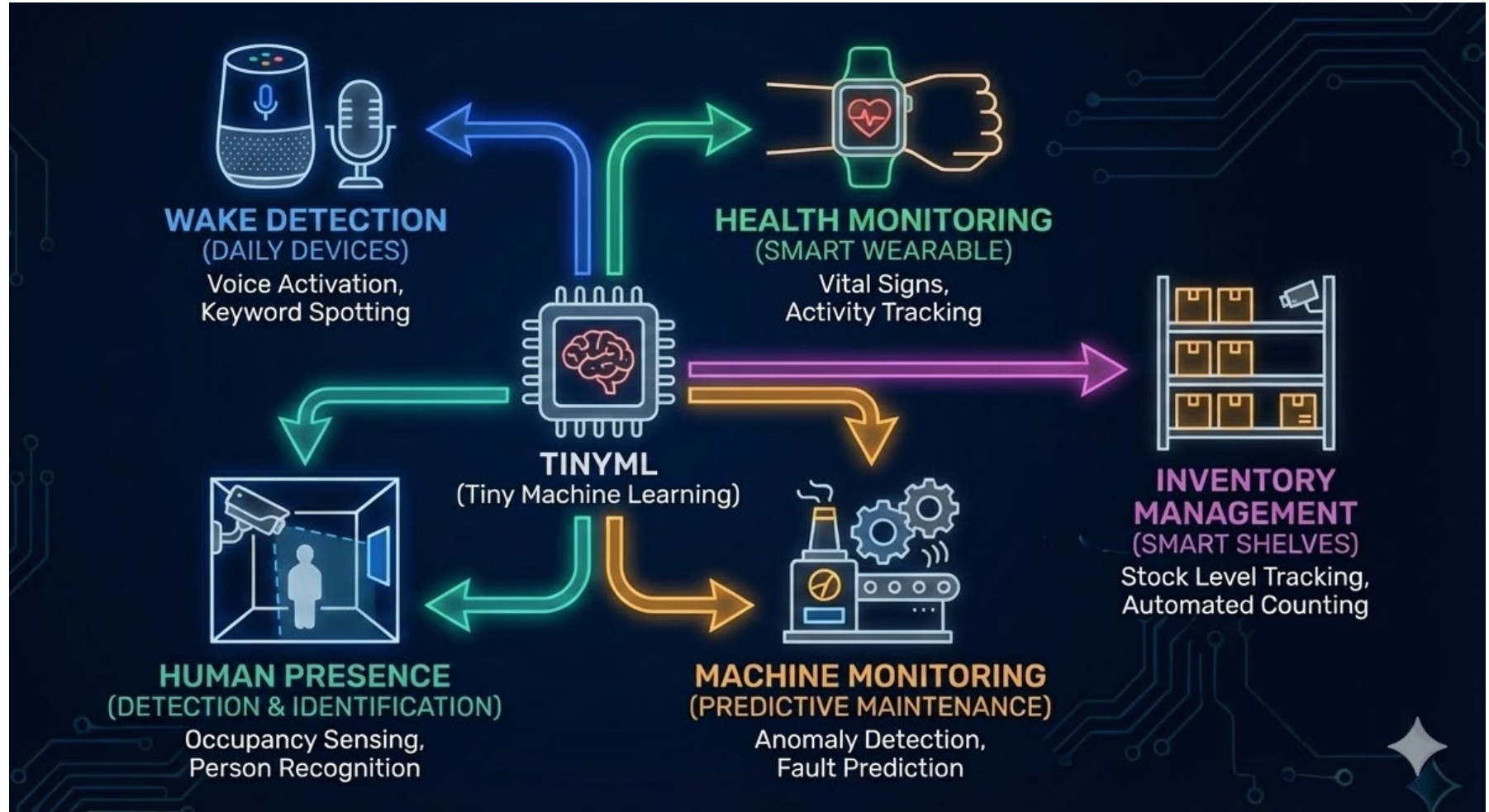
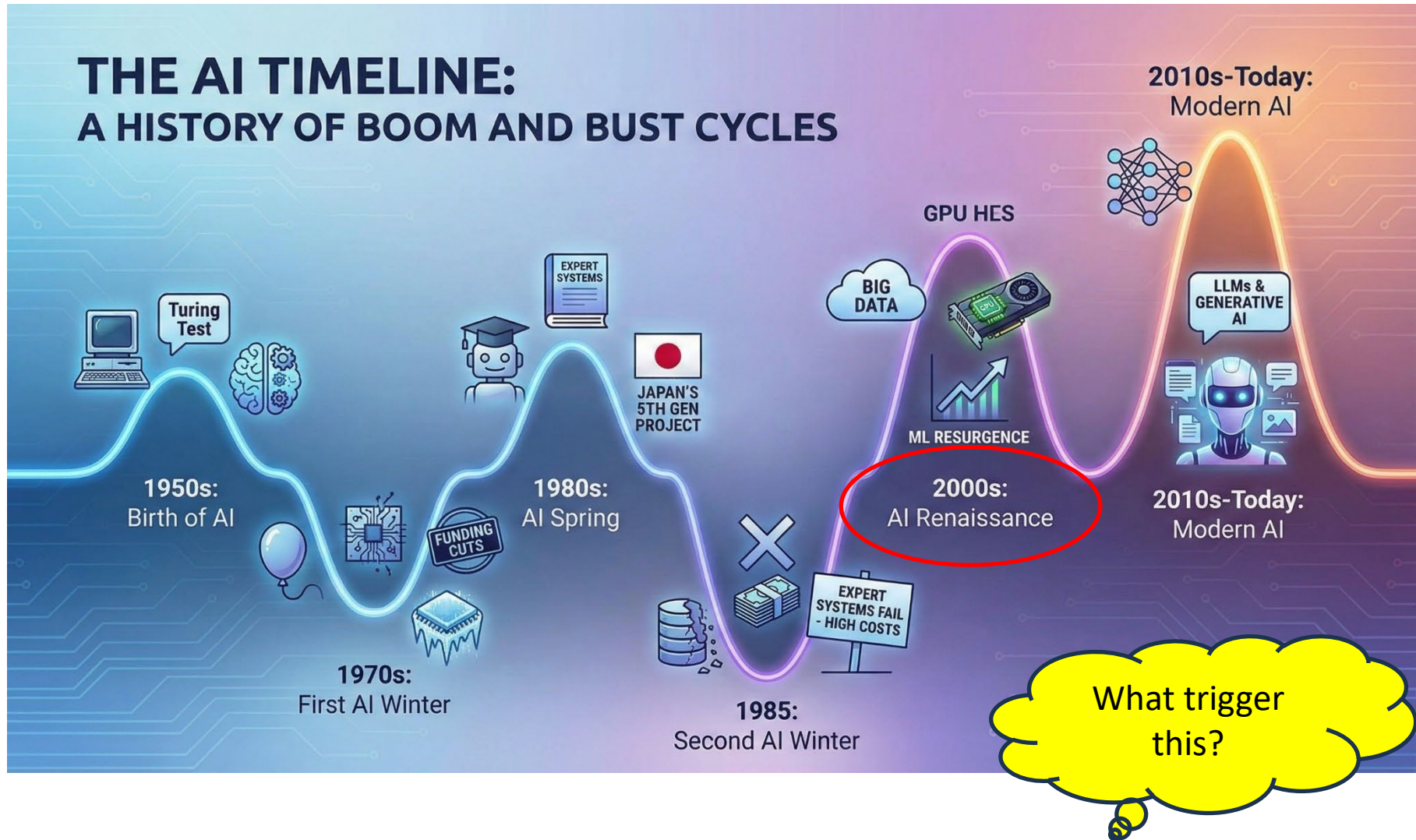


Image Src: Google Nano Banana

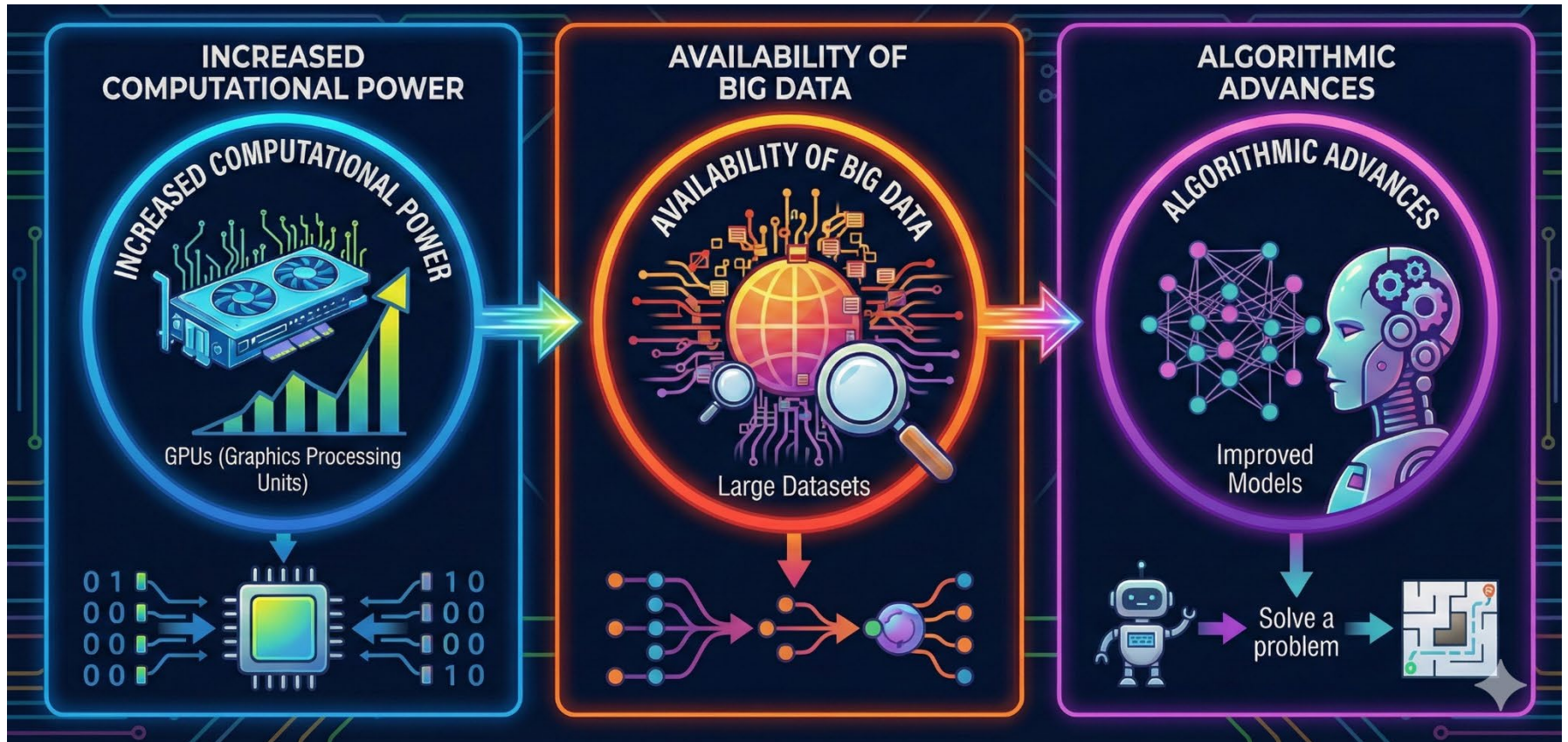


Small bites on Artificial Intelligence ...

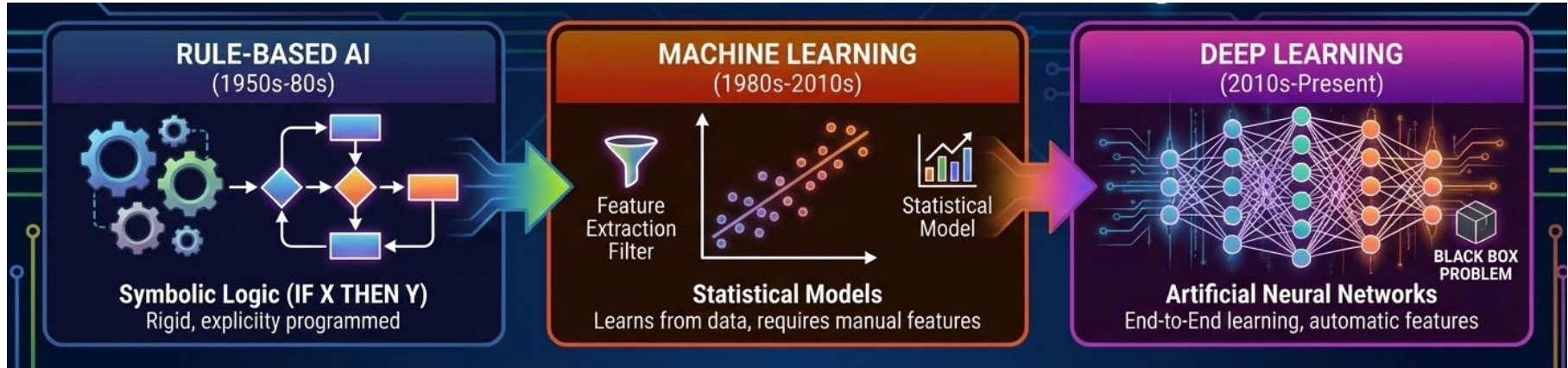
AI History Timeline



Key Drivers of the AI Renaissance



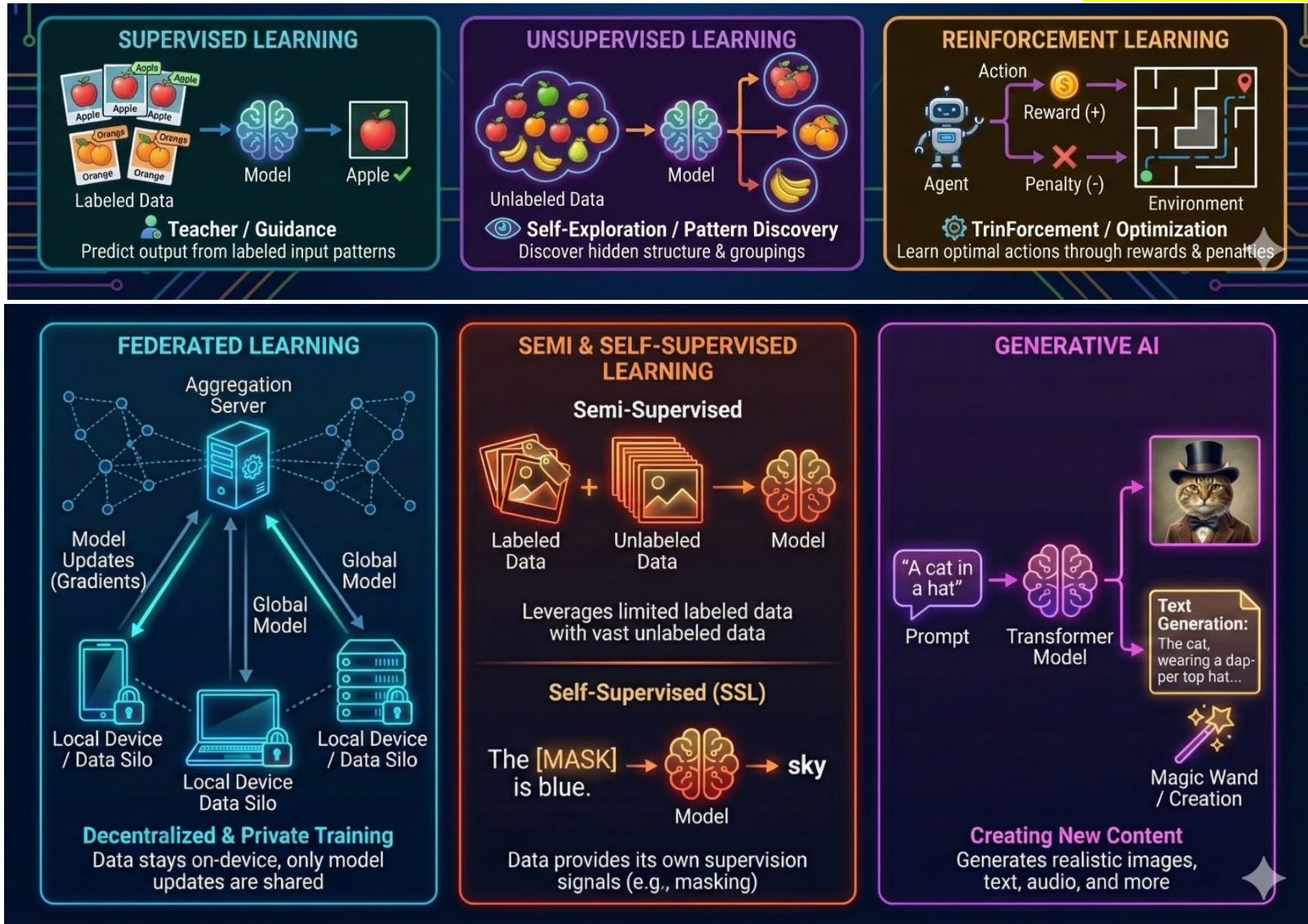
AI Evolution Timeline



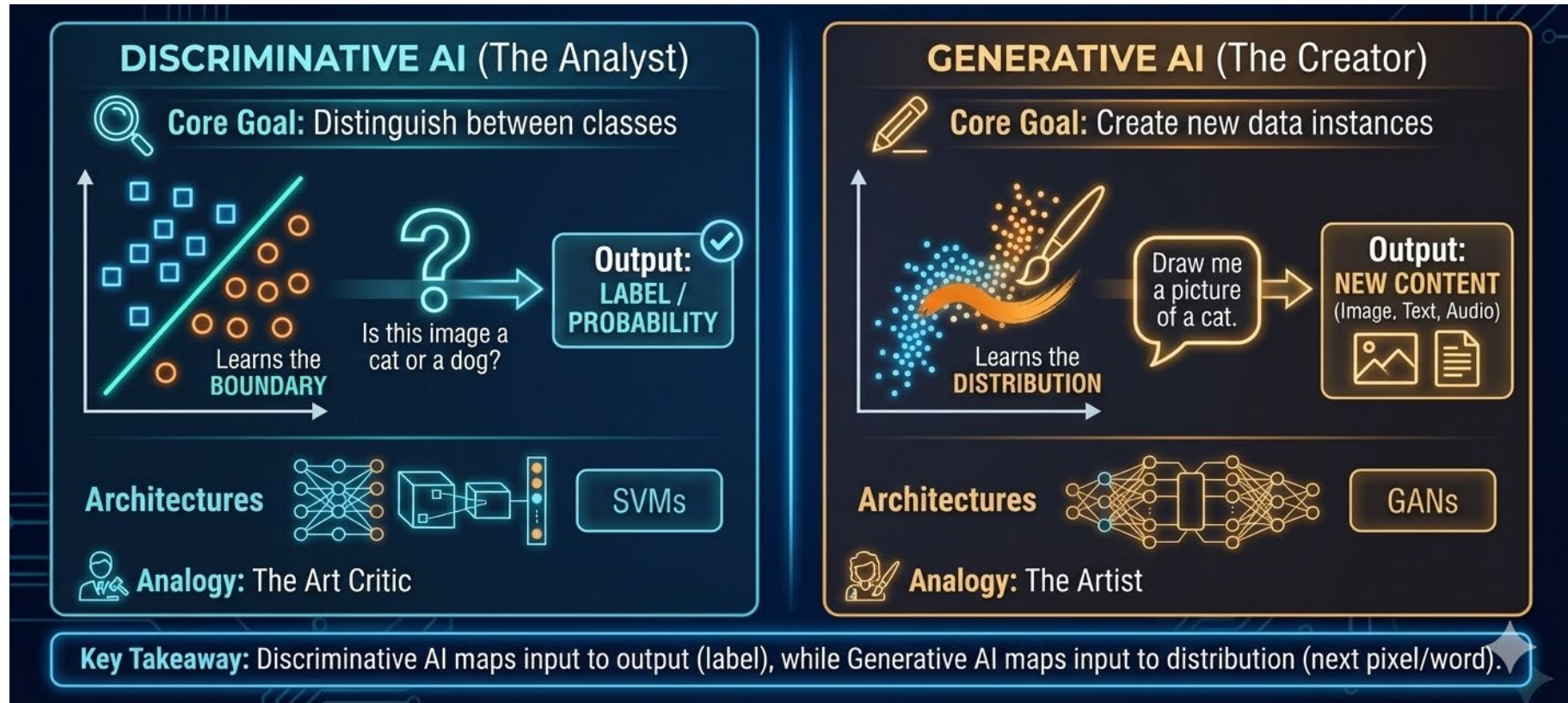
- **Rule-Based Systems.** Explicitly program the "**knowledge**" into the system using logic and rules.
- **Machine Learning.** Core philosophy shifted from **programming logic** to **programming optimization**. Instead of telling the computer *what* to know, the machine is given a statistical model and let it learn the rules from data. Requires manual feature engineering
- **Deep Learning.** Artificial Neural Networks that **learn features automatically**, compared to old ML way where features needs to be manually designed.

Common Paradigms/Methodologies ...

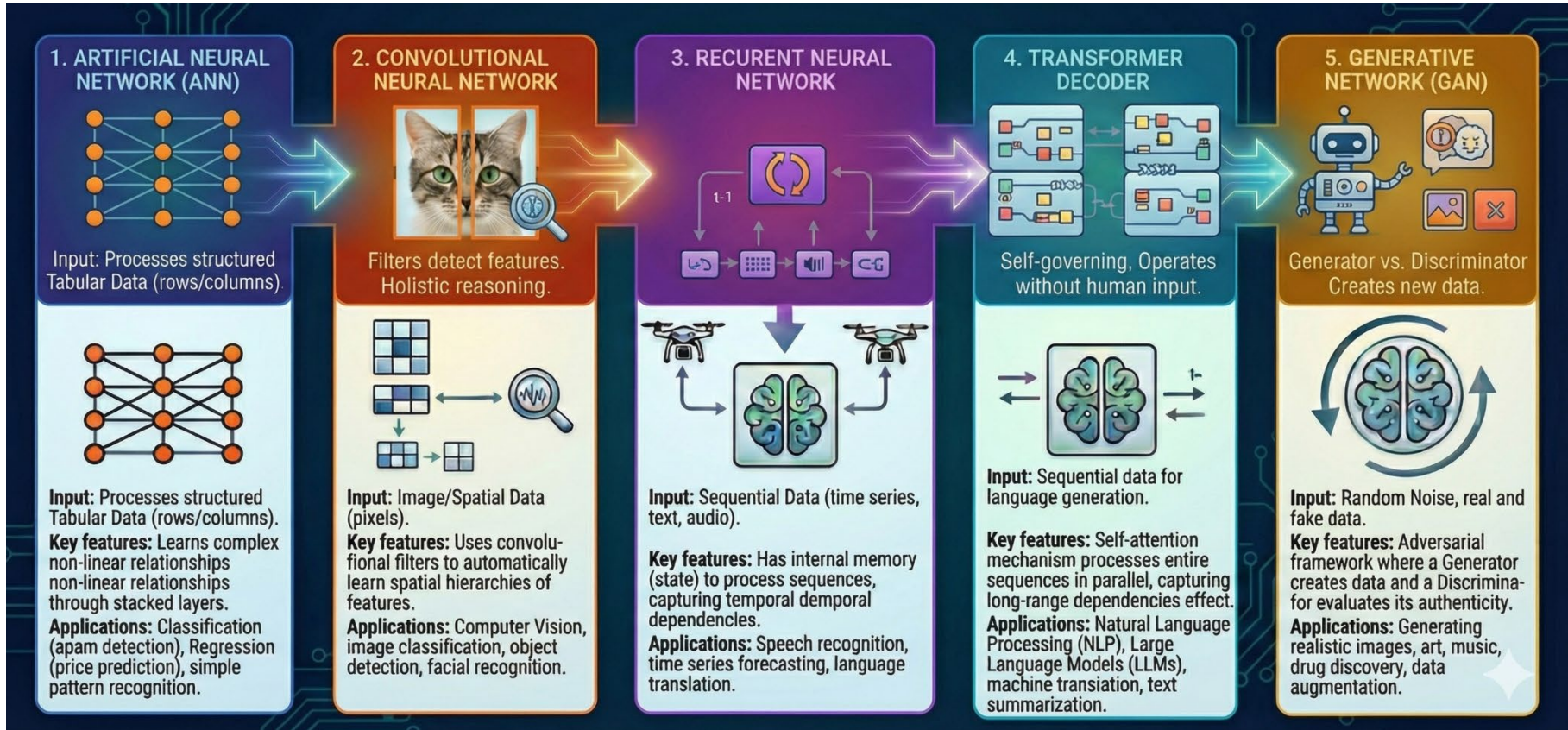
Some banana typos ...



Discriminative vs Generative AI

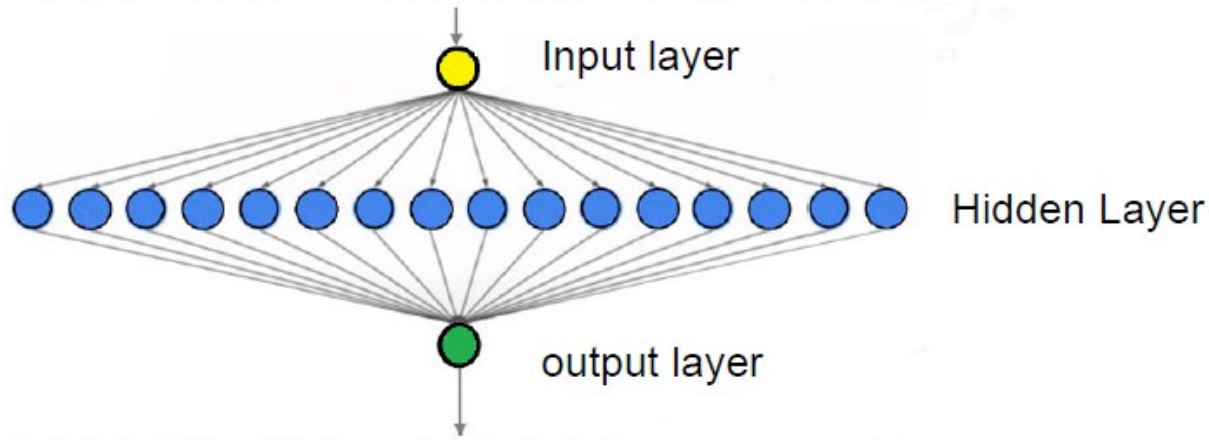


Core Deep Learning Architecture



DNN AND CNN BASICS

Neurons and Neural Network



Hidden Layer

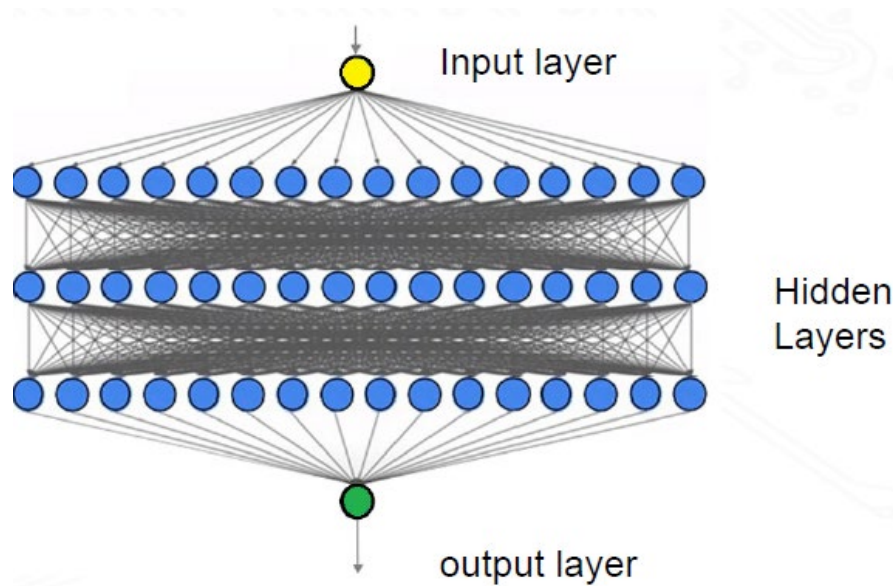
- consists of learnable parameters (the neurons)
- the 'algorithm' that can learn and improve by itself

Mimics the human brain to recognize pattern

- ability to learn

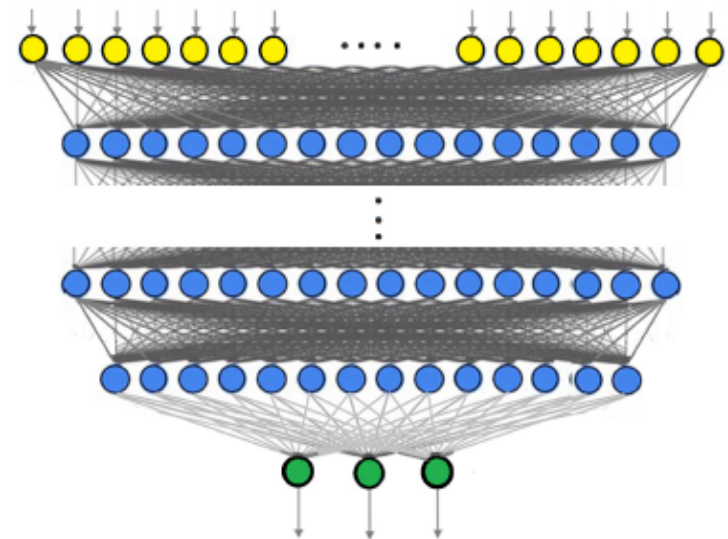


Deep Neural Network for Deep Learning

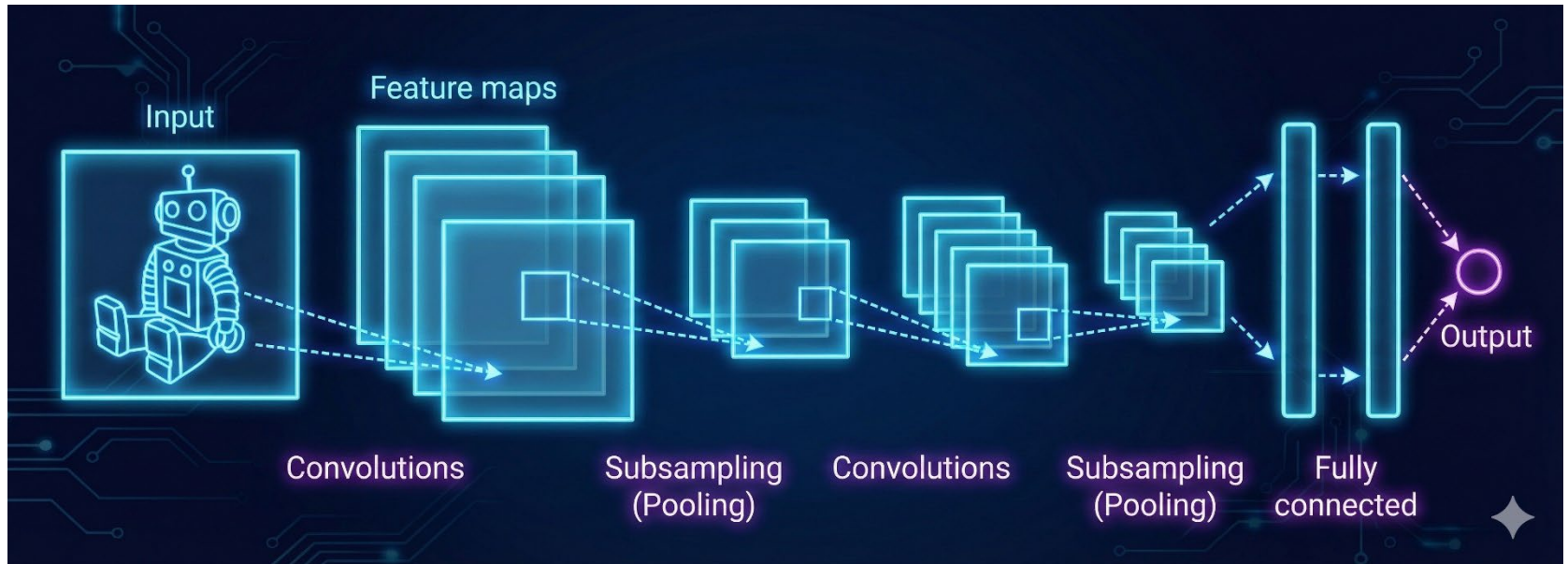


Deep Neural Network

- multiple layers of hidden layers
- much more sophisticated algorithms can be learnt

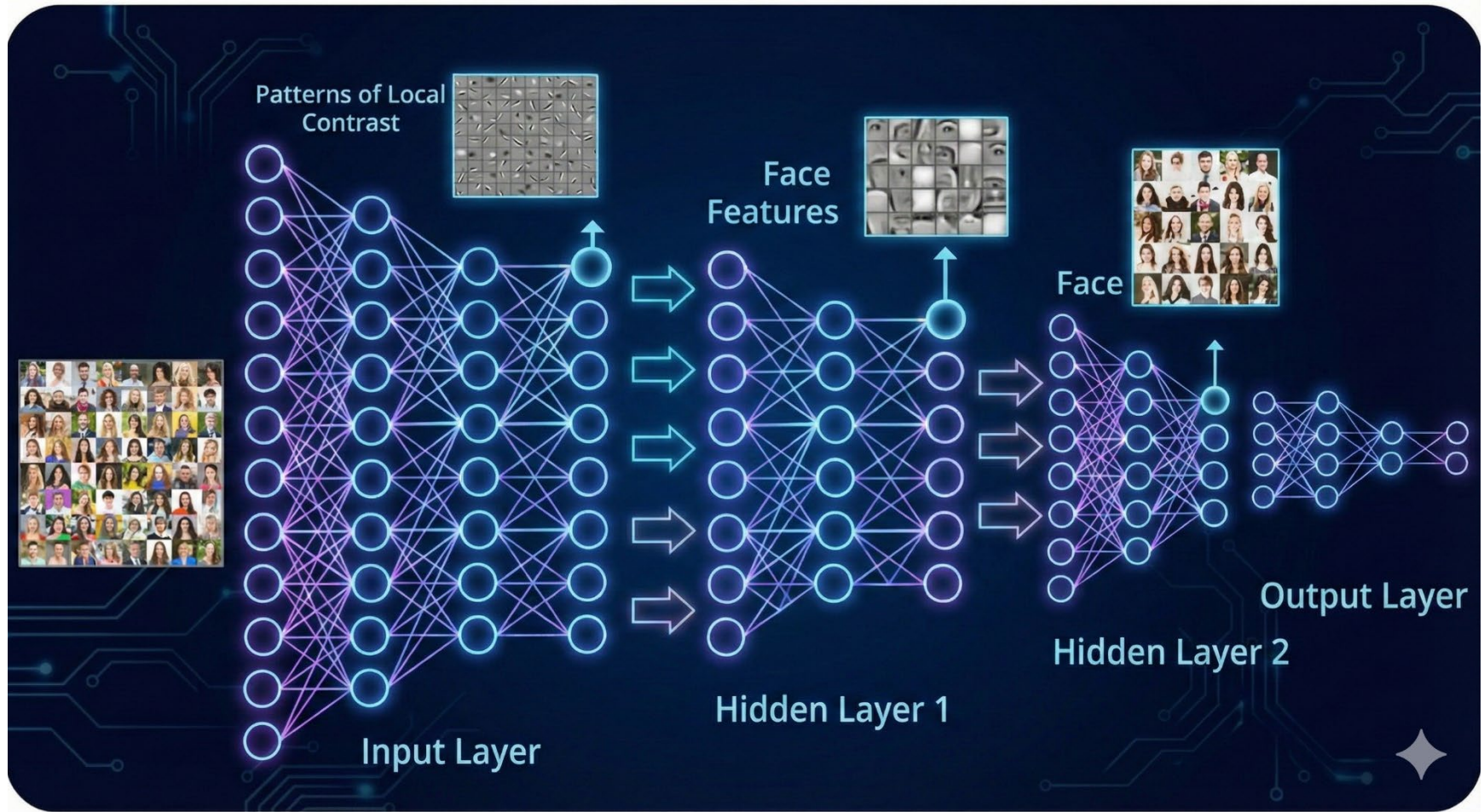


CNN for Image Recognition



- More details later ...

Face Detection Training and Inference



Internet of Things (IoT)

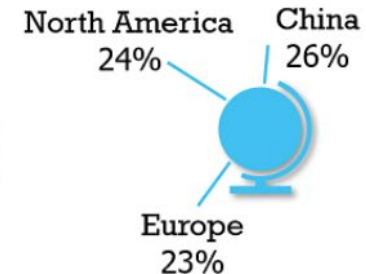
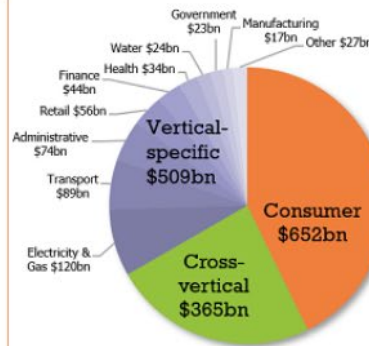
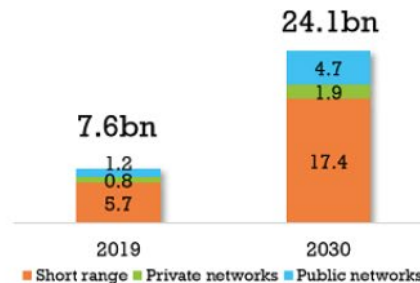
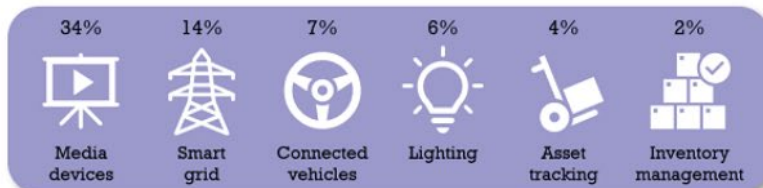
The Internet of Things (IoT) Market 2019-2030

24.1 billion

IoT connected devices in 2030 (7.6bn 2019)

\$1.5 trillion

IoT revenue in 2030 (\$465bn 2019)



Source: Transforma Insights

**Latest: 1.2 – 1.24 billion mobile phones shipped worldwide in 2024 (IDC).
And 18.8 billion IoT devices shipped world wide in 2024 (IoT Analytics)**

IoT key requirements

- Need to be very **energy efficient** such that it can be **deployed anywhere without any maintenance**.
- But **transferring of data** through network
 - requires the most **energy**
 - incur **latency**
- **Excessive capturing** of data
 - requires **more memory** on site
 - Result in larger data transfer
- Hence need to operate more intelligently
 - by **processing and analysing the data locally** without transmission
 - enable better **data privacy**
- Is it possible to incorporate ML (Deep Learning) on memory constrained microcontroller?

Memory bound and Compute bound

- Application computations performance can be memory bound or compute bound
- Memory bound
 - time taken to complete a computation depend on the memory size and speed
- Compute bound
 - time taken to complete a computation depend on the CPU speed
- Deep Neural Network performing Inference
 - mainly doing multiplication of matrices
 - same data are used repeatedly in different combinations
 - low memory requirements (tens to hundreds of KBs of RAM) i.e., mainly compute bound
 - well-suit for latest generation of 32-bit microcontrollers



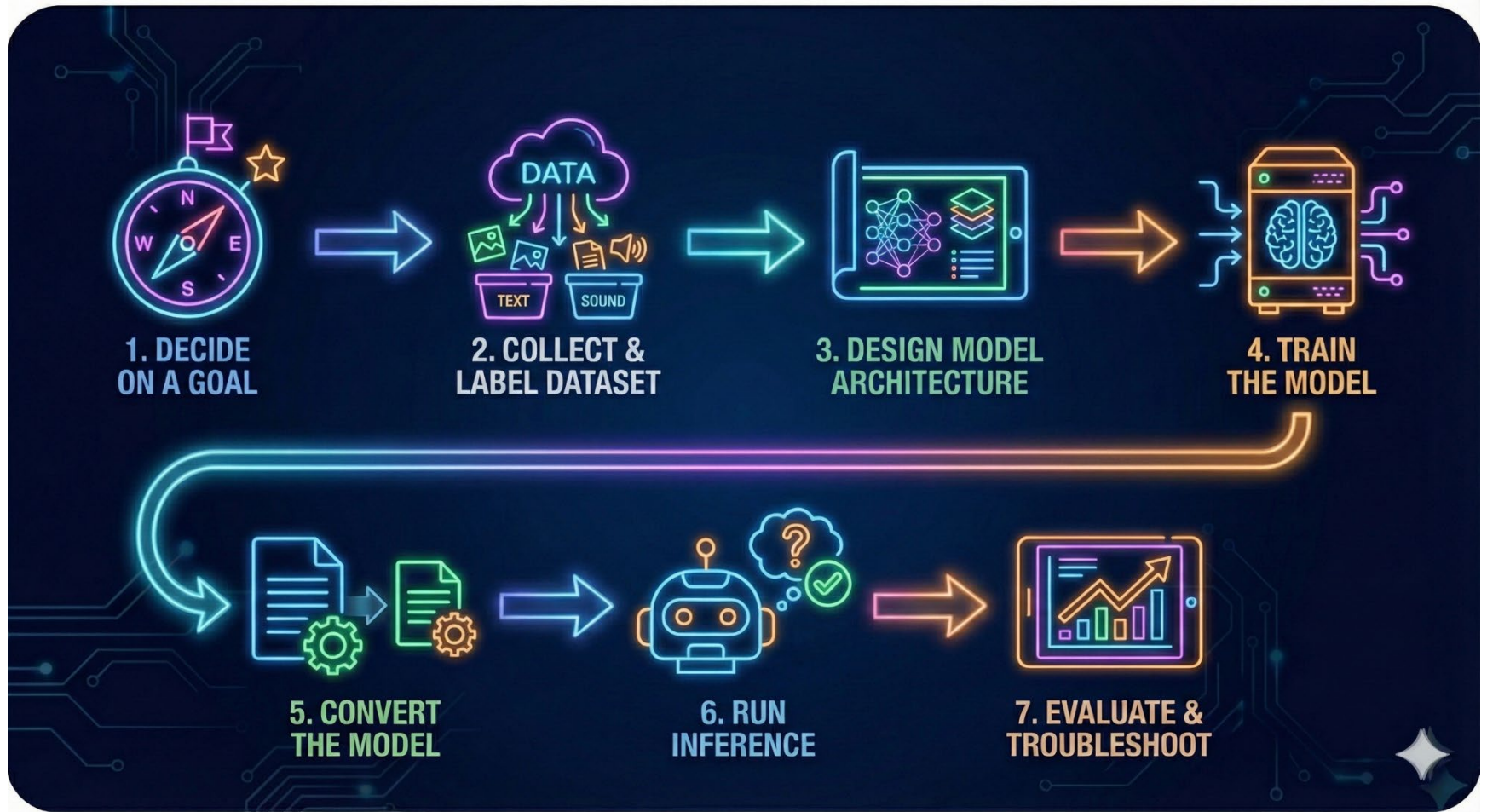
Source: Coursera images

Deep Learning Workflow

Training and Building a Deep Learning based Model

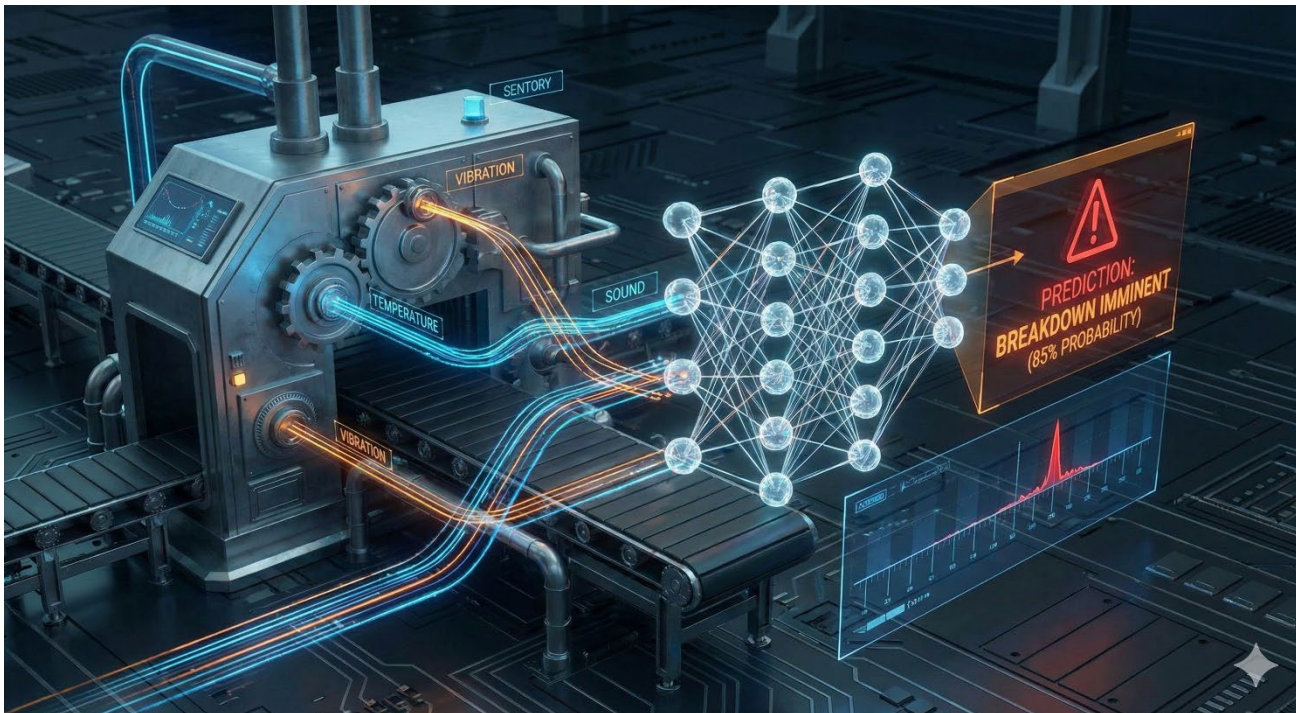
- The following are typical steps involved in developing a model based on machine learning (deep learning)
 1. **Decide on a goal**
 2. Select, collect and label a dataset
 3. Design a model architecture
 4. Train the model using the dataset
 5. Convert the model
 6. Run the inference
 7. Evaluate and troubleshoot

Training and Building a Deep Learning based Model



Case Study

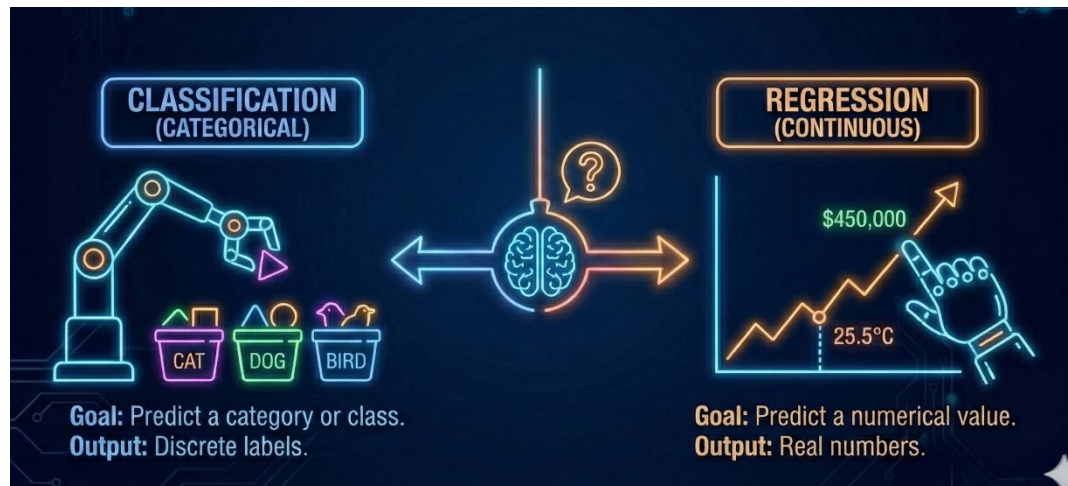
- We will use the task of “**Predicting whether a factory machine is about to break down**” as a case study to illustrate operations and considerations involved in some of the steps for model training.



Decide on a goal

Goal Setting

- Need to first decide what we want to **predict** to enable us to decide
 - what **dataset** to collect
 - which **model** architecture
- One of the basic decision to make is whether this is a **classification or regression problem**
- The factory machine evaluation is **classification problem** with two possible outcome
 - Normal
 - Abnormal



Selecting Dataset

Data Management

Know the context
of the problem

- What data to collect?
 - only **select** those that are **relevant**
 - **ignore** obvious **irrelevant** data
- **Example**: Predicting whether a **factory machine** is about to **break down**
 - **Temperature** of the machine – **Yes**
 - **Vibration** of the machine – **Yes**
 - **Noise emitted** by the machine – **Yes** (but may be related to vibration)
 - **Speed** of operation - **Maybe**
 - Duration from **last service** – **Maybe**
 - **Ambient noise** in the factory – **Probably No**
- Usually need **domain expertise** and perform some experiments to decide, and need to consider ease of collecting the data

May be fuzzy
at times

Collecting Data

Data Management

- After deciding on the dataset to collect
 - how much data to collect?
 - rule of thumb - the **more the better**.
- Collect data that potentially related to each **model of failure** and in **full range of conditions**.
 - Full range of conditions mean data need to be collected in different time of the day, weather and any other external factors that may affect the data.
- Collect as **a set of time series data to identify potential correlation**
 - vibration level every **10 seconds**
 - temperature **every minute**
 - rate of production once **every two minutes**
- Note that different dataset may have **different sampling rate** so data may need to be transformed into an appropriate form for the model.

Labelling Data

Data Management

- After collecting the dataset
 - label the data as **normal** and **abnormal**
 - As the task is to predict if a machine is **about** to break down, we assume 'just before' failure = **abnormal**, will need some trial and error in practice to identify the 'abnormal' data
- Example: Predicting whether a factory machine is **about to break down**

Table 3-1. Data sources

Data source	Interval	Sample reading
Rate of production	Once every 2 minutes	100 units
Temperature	Once every minute	30°C
Vibration (% of typical)	Once every 10 seconds	23%
Label ("normal" or "abnormal")	Once every 10 seconds	normal

Source: "TinyML" by Pete Warden & Daniel Situnayake

Which Model Architecture to use

Model Design

- Many possibilities, depending on
 - type of problem
 - type of data available (e.g. time series)
 - data transformation (e.g. in frequency domain, statistical derivatives etc)
 - personal experience (domain expertise)
- Also consider the resource constraint of the device to deploy the model
 - memory size limitation
 - processor capability
 - microcontroller special features (e.g. hardware accelerator)
- Start with a simpler model (less layers of neurons)
 - iterate until we obtain a useful result

Features and Window

Model Design

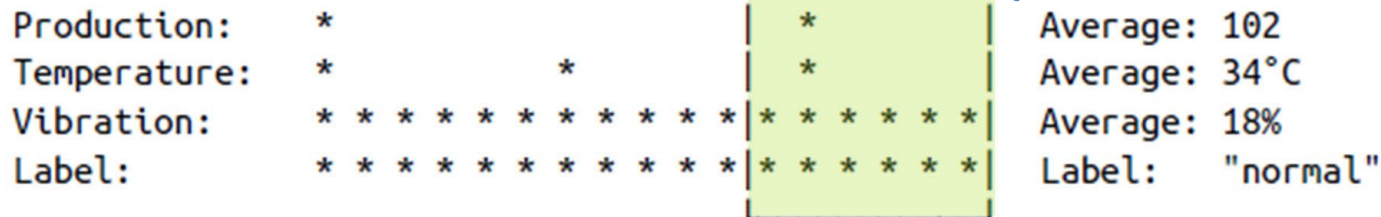
- Feature

- information that the model will accept for training
- a 1-D tensor with 3 entries may represent 3 features
- 'images' may be represented with higher dimension tensors

- Example: **Factory Machine**

- Feature = [Production, Temp, Vibration] = [102, 34, 0.18]

1-min Window



Source: "TinyML" by Pete Warden & Daniel Situnayake

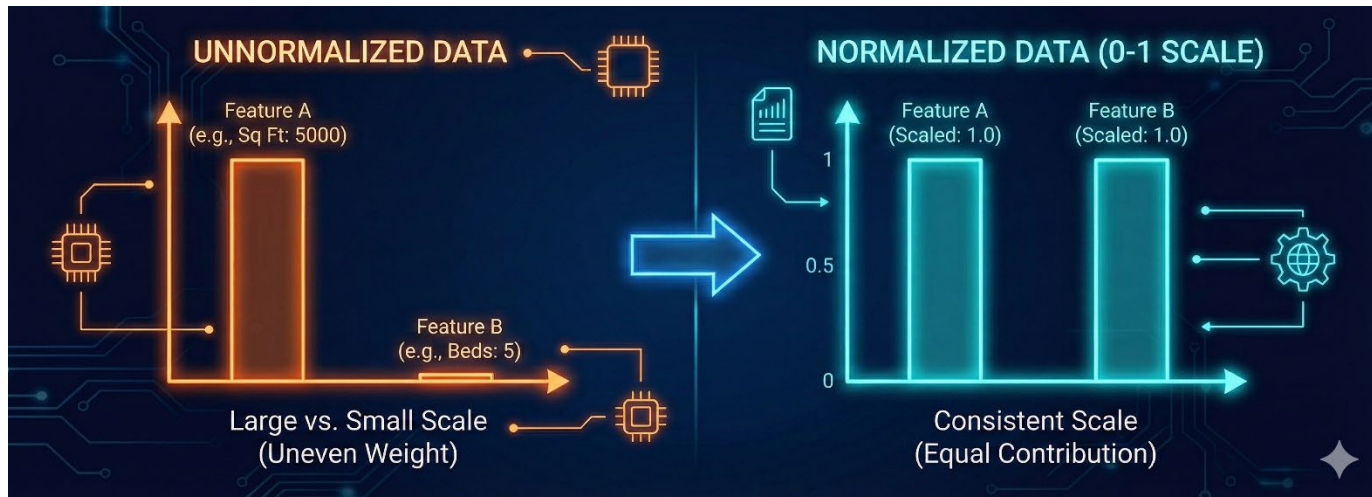
- The feature contains **time-series of unequal interval value**
 - choose a **Window of 1 minute** and take their **average values** during the interval
 - take the **most recent** value for any that lack a datapoint in the current window

Data source	Interval
Rate of production	Once every 2 minutes
Temperature	Once every minute
Vibration (% of typical)	Once every 10 seconds
Label ("normal" or "abnormal")	Once every 10 seconds

Normalization

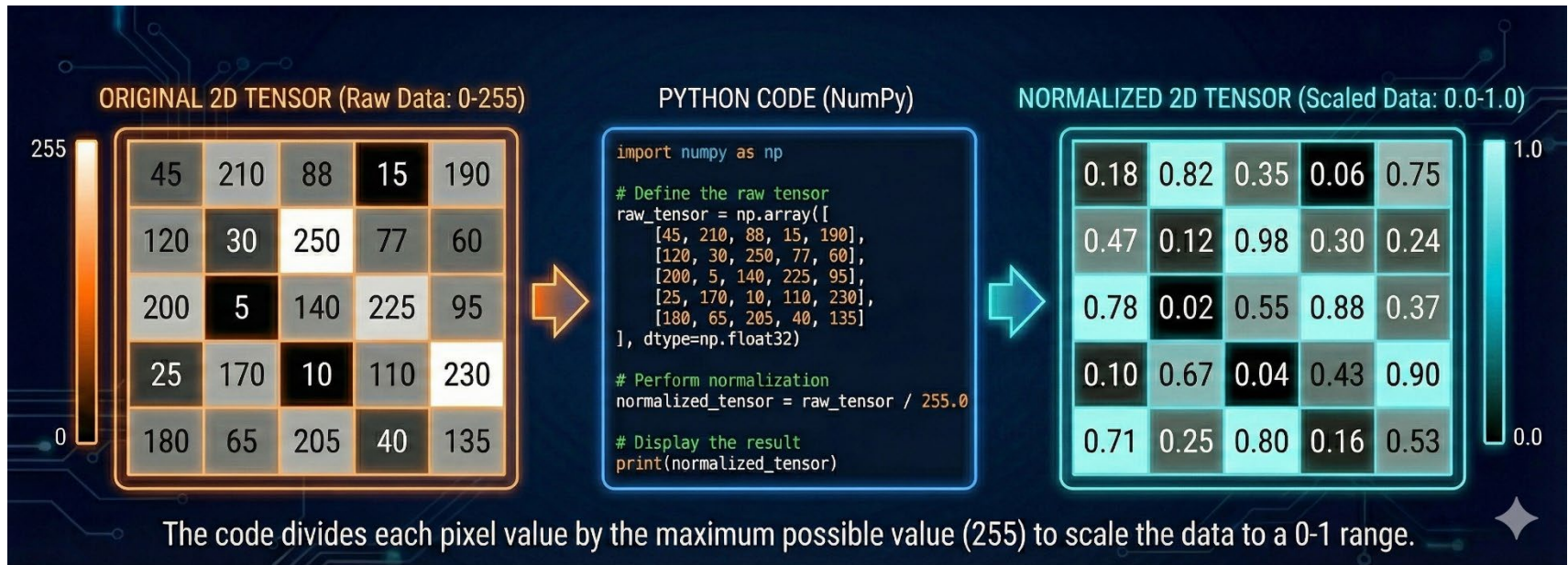
Model Design

- Different input features typically have values of different orders
 - Results in uneven contribution from each feature during model training.
 - Affects the rate of convergence and accuracy.
- Normalization
 - process of rescaling numeric data to a standard range (typically 0 to 1 or -1 to 1).
 - Ensure all input features contribute equally to the model's learning process, regardless of their original units or magnitude.



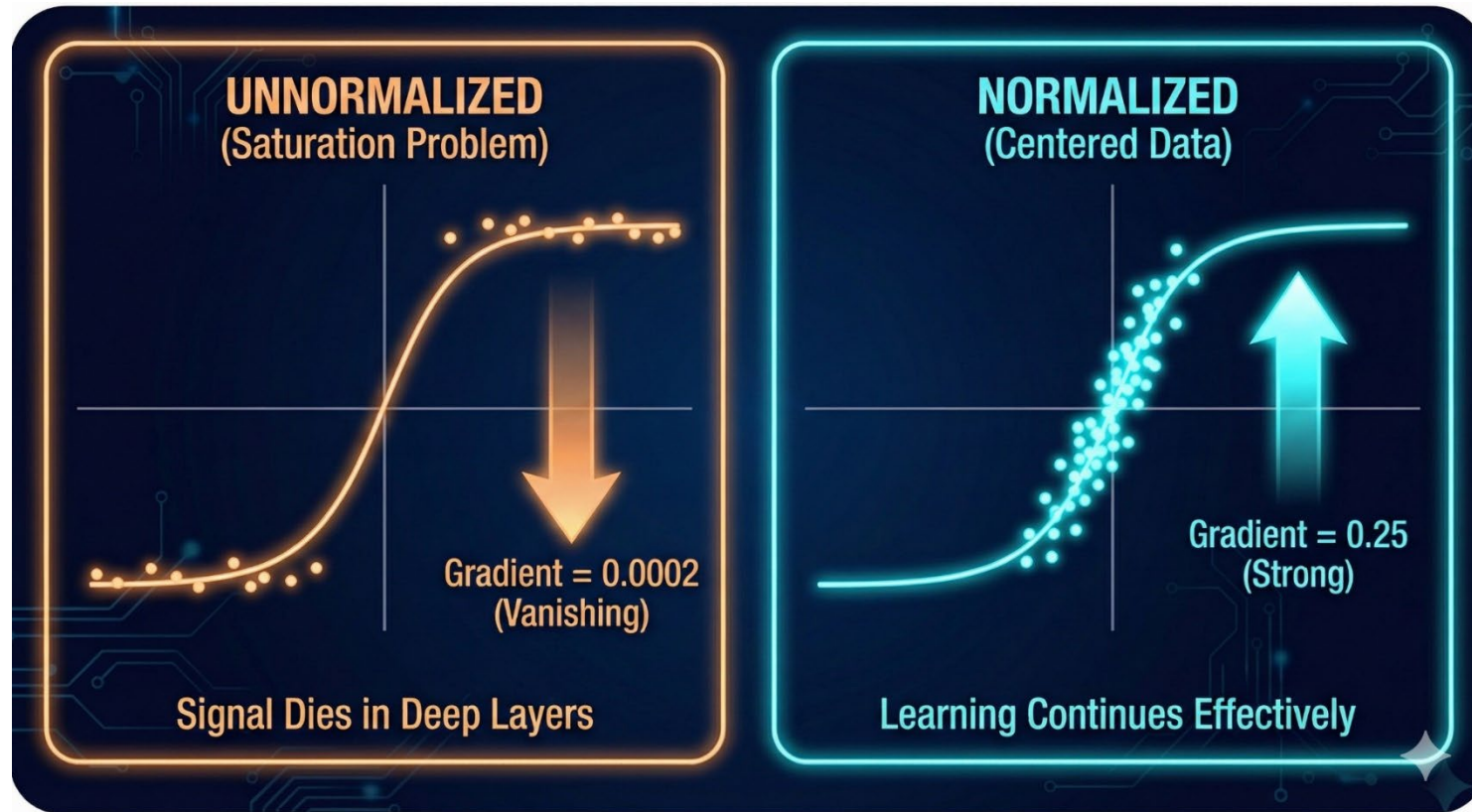
Normalization Example

Model Design



Normalization – Vanishing Gradient

Model Design



- **Backpropagation** used in model training relies on **chain rule** which **multiplies gradients in adjacent layers**.
- Small gradients result in even smaller product. When product is close to zero, early layers stop learning. More details in later chapters.

Training the Model

Model Training

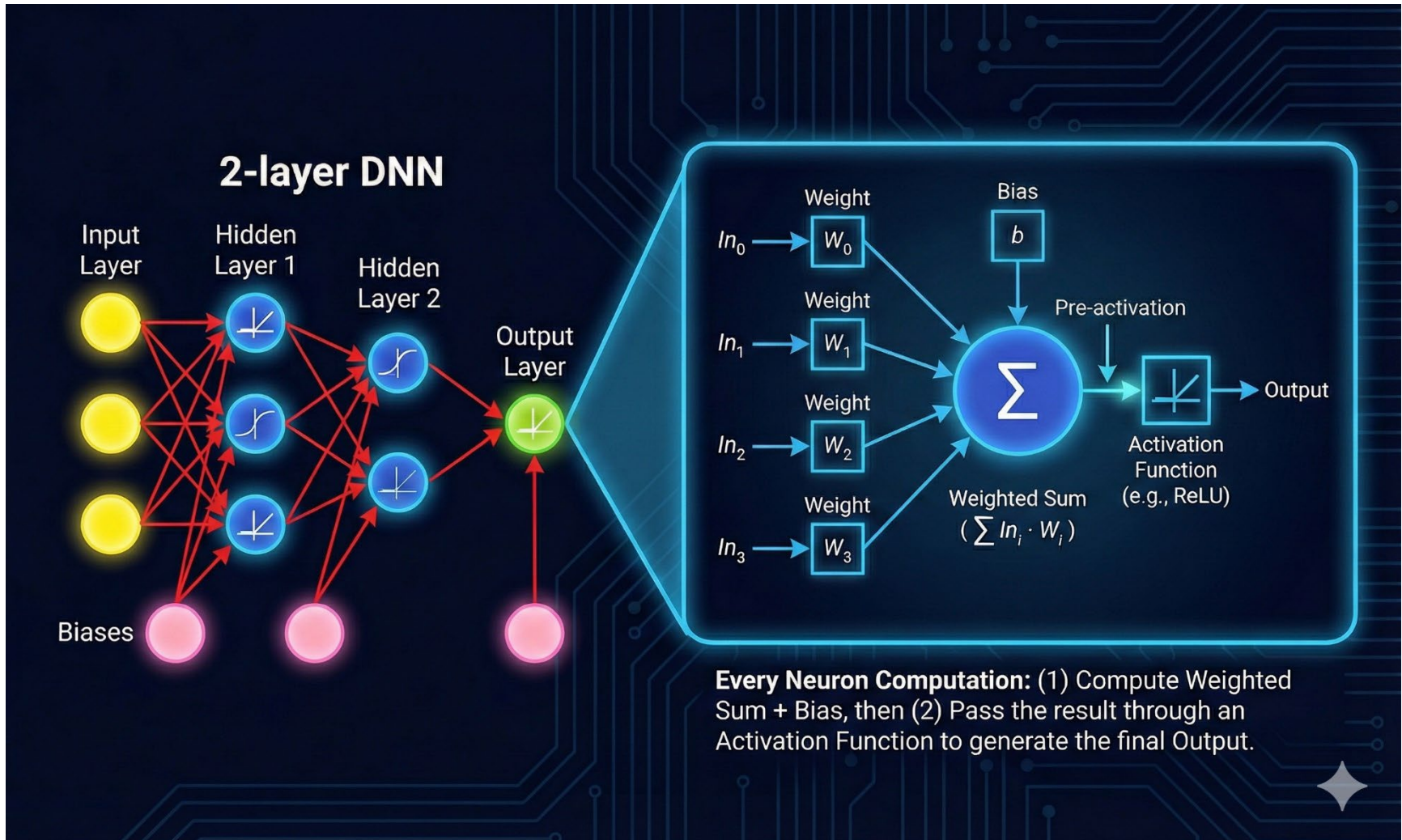
- The process for the model to learn to produce the correct output for a given set of input
- Training data are input into the model
 - model predicts the output value
 - compare the predicted output against the expected output
 - calculate the error
 - make small iterative adjustment to the model's parameters until the error is acceptable (Backpropagation)
- Each neuron in the neural networks contains two parameters
 - weight and bias

Forward

Backward

Neural Network Architecture

Model Training



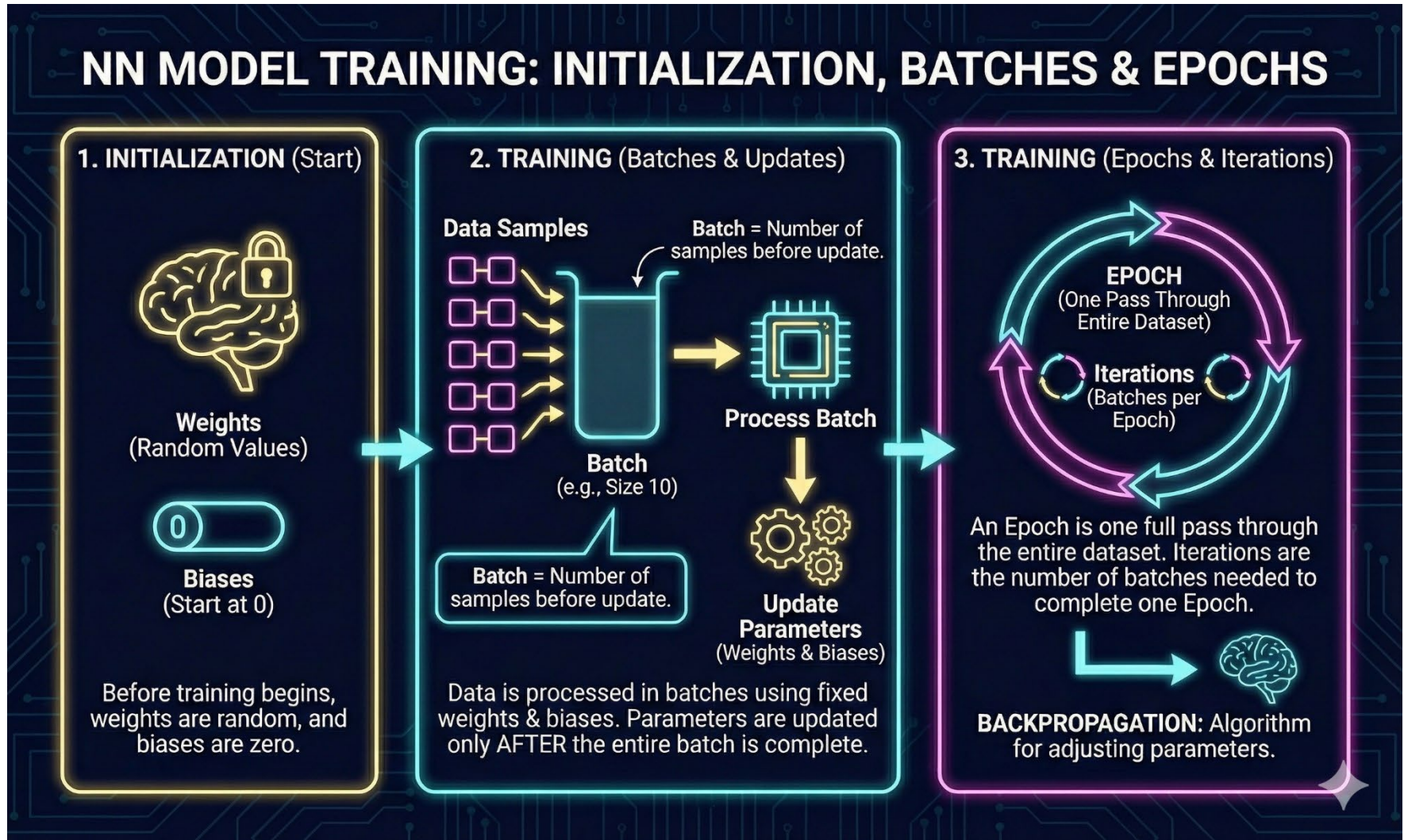
Terms used in Model Training

Model Training

- At the beginning
 - **weights** are assigned with **random** values
 - **biases** typical start with a value of **0**
- During training
 - data are fed in **batches** into the model
 - ‘**batch**’ is the number of data to be processed **before updating the parameters**
 - E.g. **Batch size of 10** means **10 samples** will be processed before model **parameters** are **updated**.
- Adjustment of parameters is done by an algorithm call **Backpropagation**
- **Epochs** is the **number of times** that the learning algorithm will work through the **entire training dataset**.
- **Iterations** is the **number of batches** required to complete one **Epoch**.

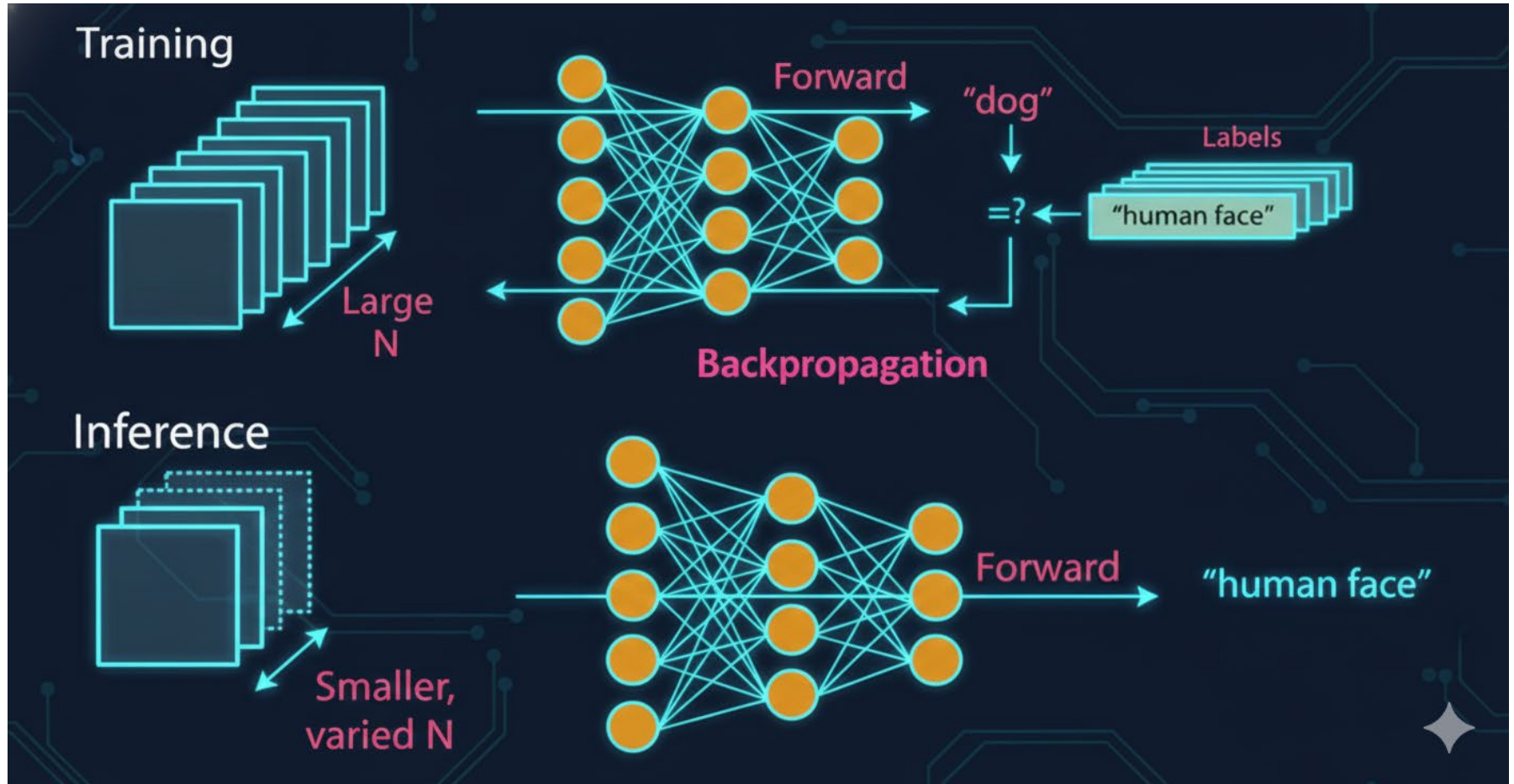
Terms used in Model Training

Model Training



ML Training and Inference

Model Training



Computation required for training is much more than that required by inference operation.

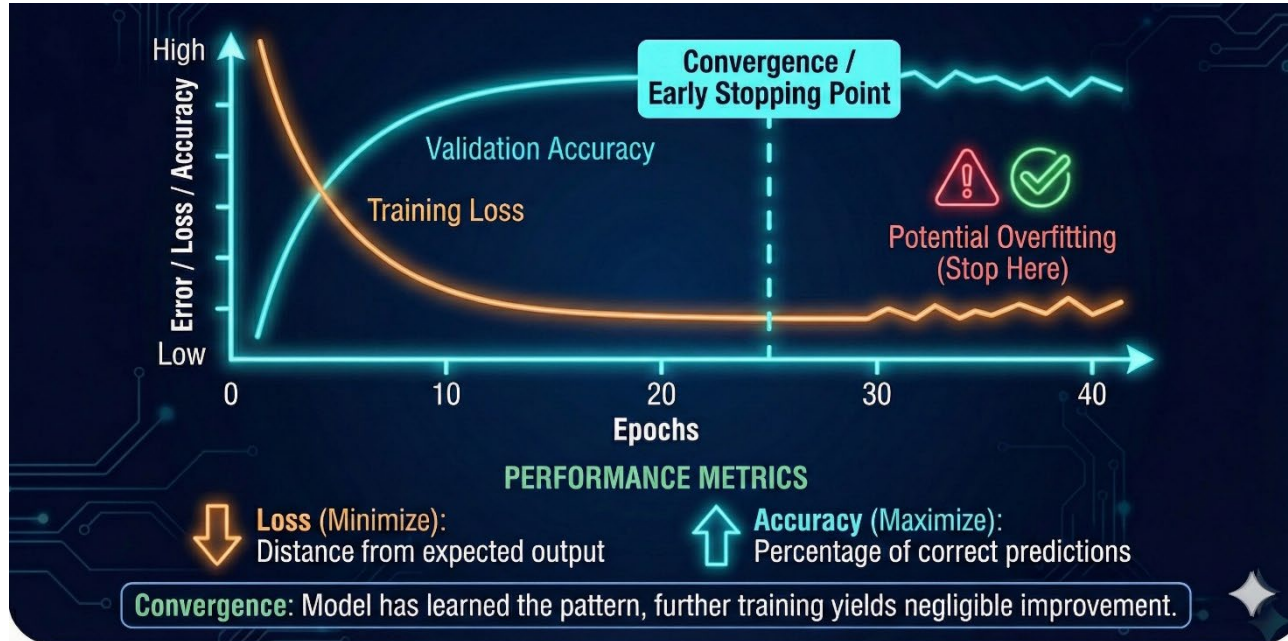
How many Epochs should we use?

Model Training

- We stop when the model's performance stop improving, i.e. Early Stopping.
 - Reduce computation wastage and prevent overfitting.
- Metrics used to evaluate the model performance
 - Loss and/or Accuracy
- Loss
 - How far the model is from predicting the expected output
- Accuracy
 - Percentage of time the model make the right predictions
- If the model output begins to make accurate prediction against expected output
 - convergence has occurred.

Loss and Accuracy Graph for Convergence

Model Training



- To **improve** the model's performance
 - change the **parameters** used to set up the model
 - change the **number of layers**
 - change (increase) the number of **epochs**
 - **More ...**

Underfitting and Overfitting

Model Training

- Two most common reasons a model fails to converge
 - Overfitting and Underfitting
- Underfit
 - High training and validation loss
 - model has not yet been able to learn to make a good prediction
 - typically, due to architecture being too small to capture the complexity of the system i.e. simple model.
 - Try to increase the complexity of the model architecture
- Overfit
 - Low training loss but high validation loss
 - model can predict perfectly based on its training data but not able to perform well for data not previously seen
 - can be due to memorizing, or 'short cut' present in the training data. E.g. recognising of texts that are embedded in the training data
 - Apply regularization techniques such as Data Augmentation, Dropout and Early stopping

Underfitting and Overfitting

Model Training

DIAGNOSING MODEL HEALTH: UNDERFITTING vs. OVERFITTING

UNDERFITTING (High Bias): Too Simple

SYMPTOM & OBSERVATION



ROOT CAUSE

Model architecture is **too simple** (too few layers/neurons); Insufficient training time.

SOLUTIONS

- Increase Complexity** (Add Neurons/Layers)
- Train Longer** (More Epochs)
- Add Relevant Features**

OVERFITTING (High Variance): Memorizing Noise

SYMPTOM & OBSERVATION



ROOT CAUSE

Model is **too complex** (too many parameters) for the data size; Memorizing noise.

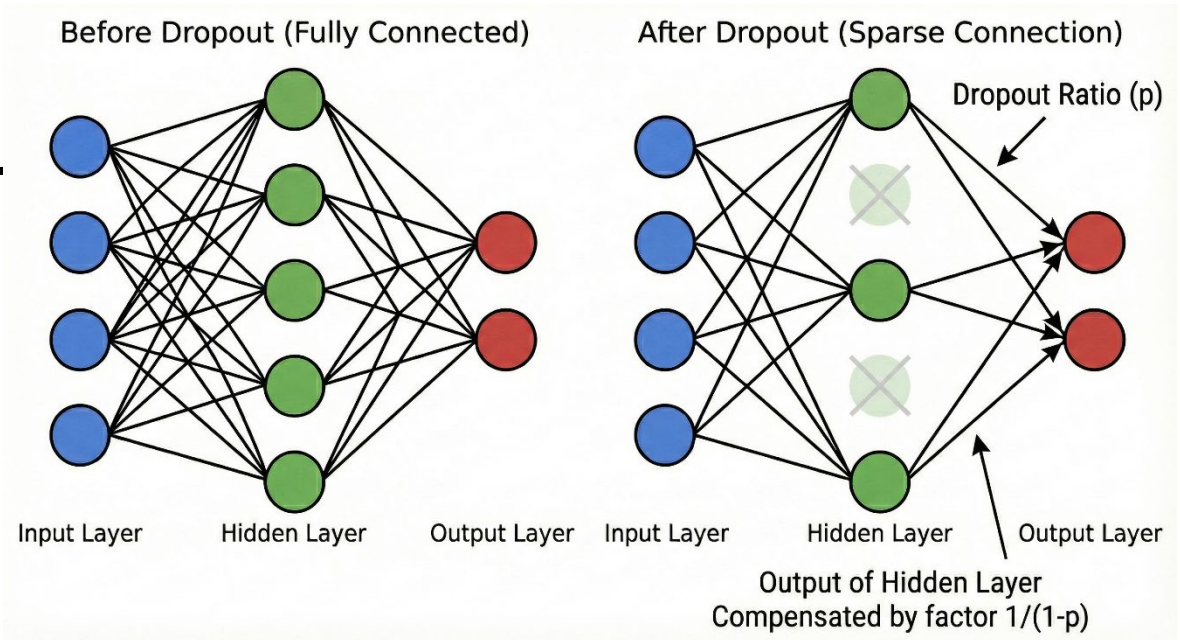
SOLUTIONS

- Early Stopping** (Stop when Validation Loss rises)
- Regularization** (e.g., Dropout)
- Data Augmentation** (Increase dataset size)

Optimal/
Sweet Spot

Dropout

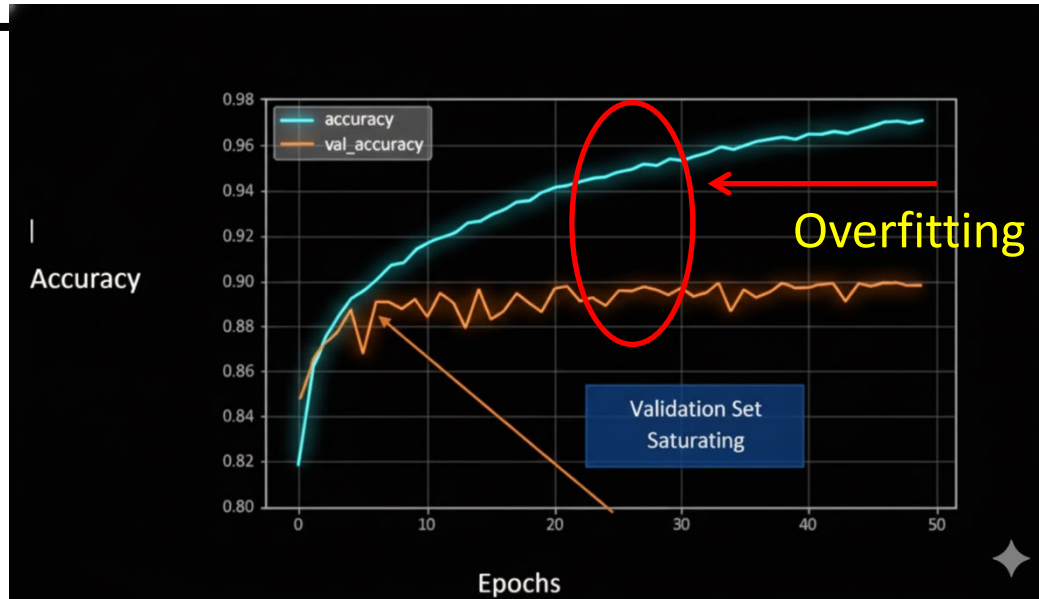
Model Training



- A certain **percentage** of node in the network are **dropped** during training of the model.
- Makes model more **robust**, able to **generalise better** and **reduces** model **overfitting**.
- In practice, **certain percentage of the input** to each node is **set to zero** during each **training iteration** where weights are updated.
- The **effect** of the **missing inputs** is compensated by **scaling** the **non-zero inputs** by $1/(1-p)$ where p is the probability an input will be dropped.
- Note that Dropout is **only applied** during **training**. **All nodes** are enabled during **inference operation**,

Early Stopping

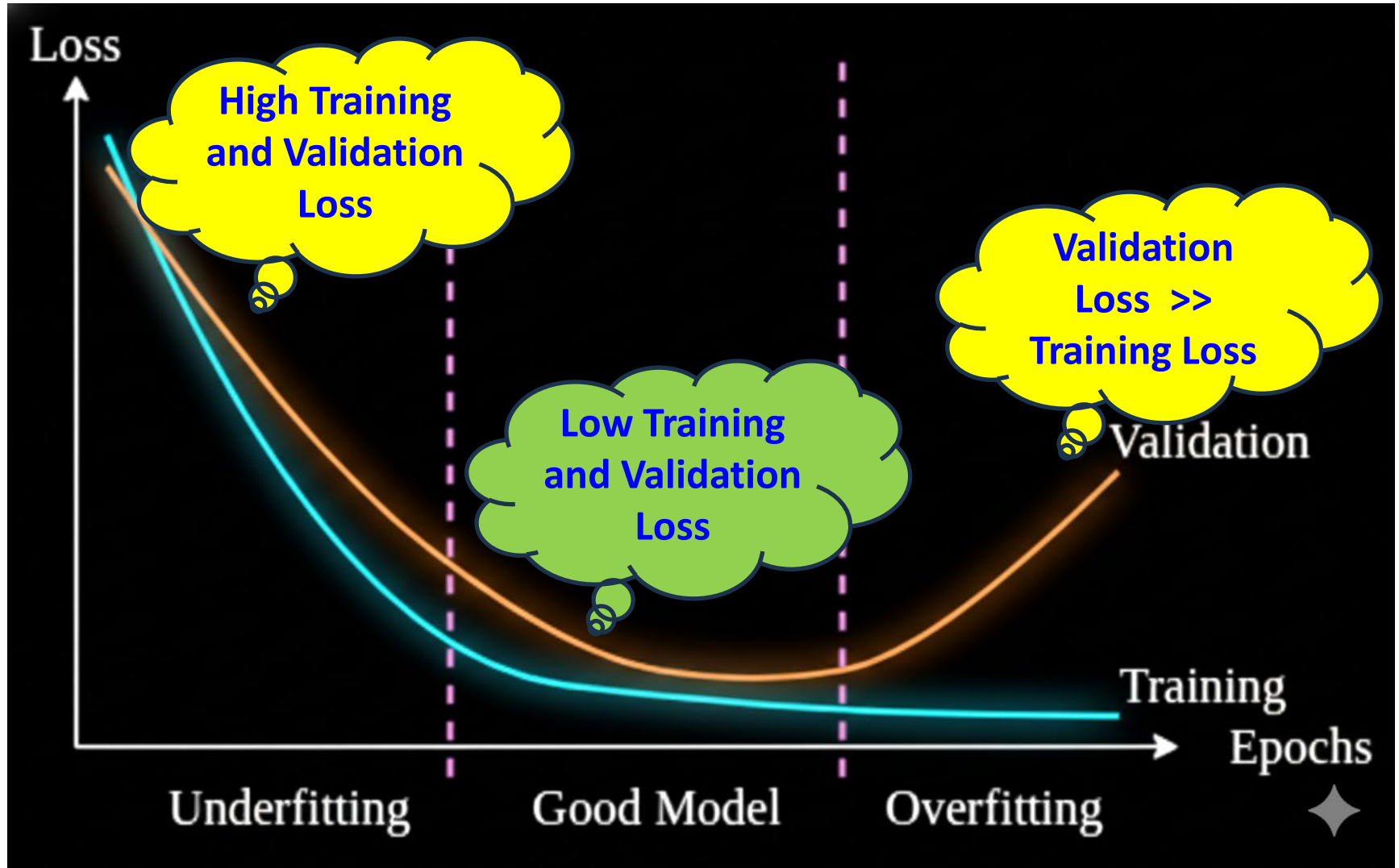
Model Training



- A regularization technique that stops the training iterations when the performance of a model stops improving.
- The performance metrics maybe accuracy, loss etc..
- It helps to reduce computation wastage and prevents overfitting of model.
- Use of 'patience' parameter to ensure that validation performance has stop improving for a few epochs before applying the early stop.
- Prevent the scenario of stopping too early and results in underfitting.

Overfitting and Underfitting Loss Graph

Model Training

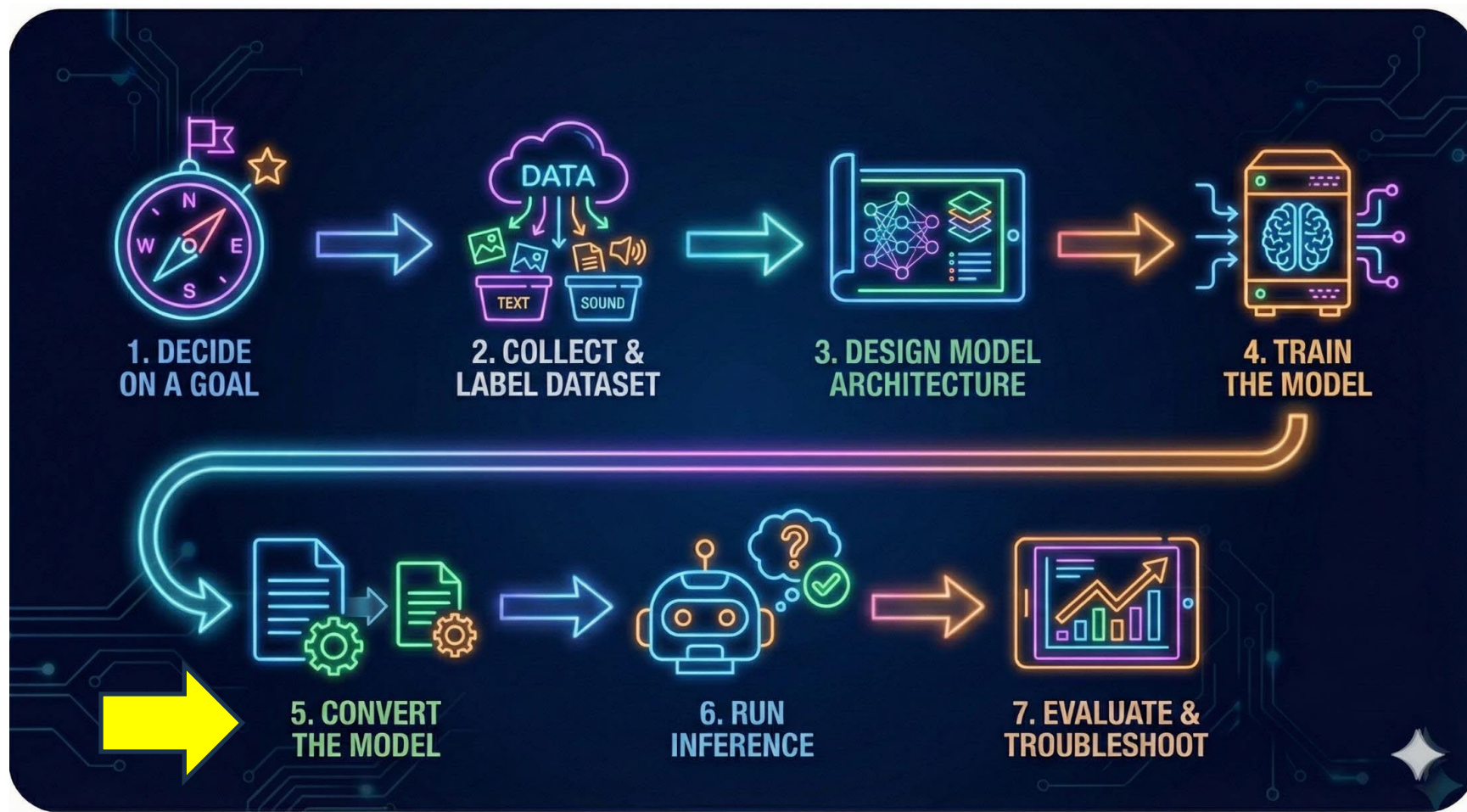


Training, Validation and Testing

Model Training

- Dataset are **split into 3 groups** during training to check the performance of the model against overfitting
 - **Training Data** (e.g. 60%)
 - **Validation Data** (e.g. 20%)
 - **Testing Data** (e.g. 20%)
- Training is done using the **Training data**
- **Validation data** is fed through the model periodically to calculate its loss
 - provide a **better representative** of the performance because the model has not seen the data before
 - **adjust the model** to improve validation performance
 - Validation data should **never** be used for training.
- Once the validation performance is acceptable
 - use the **test data to do final confirmation**

Training and Building a Deep Learning based Model (Recap)



Converting the Model for Microcontroller

Model Conversion

- Normally, the model can be deployed once the Training is completed
 - the **model is loaded into the memory** and executed by a **TensorFlow interpreter**
- But TensorFlow interpreter is **too big** for **microcontroller** used in TinyML
 - need a **lightweight** version of the **interpreter** and tools
 - **TensorFlow Lite**
- Need to **convert** the TensorFlow based model to the **TensorFlow Lite** format with the TensorFlow Lite Converter tool

Deployment - Running Inference

Running Inference

- The **tensorflow lite model** can now be **integrated** into the **application programme**
 - using the TensorFlow Lite for Microcontroller library
 - C++ programming language.
 - More details in later chapters.
- Example: Predicting whether a factory machine is about to break down

Normal score	Abnormal score	Explanation
0.1	0.9	High confidence in an abnormal state
0.9	0.1	High confidence in a normal state
0.7	0.3	Slight confidence in a normal state
0.49	0.51	Inconclusive result, since neither state is significantly ahead

Source: [MVA-L](#) by Petete Warden & Daniel Situnayake

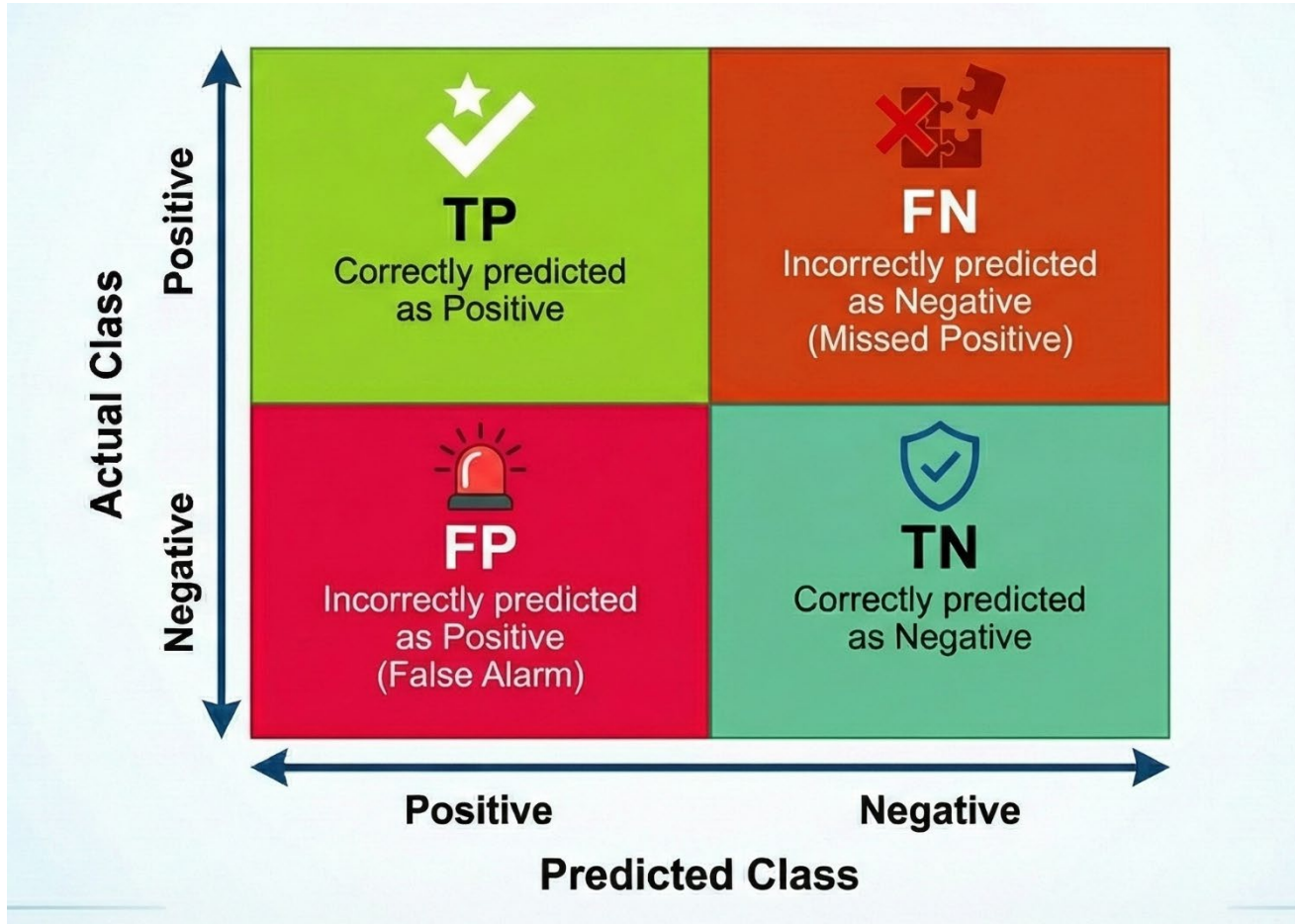
Field Run and Evaluation

Model Evaluation

- Does it **work to expectation**?
- If not, will need to further tweak the model.
 - Some examples of possible cause of model inaccuracy and corresponding actions
 - **dataset** used in training is **not** exactly **representative** of data in real operation (e.g. due to ambient temperature)
 - collect new data when operating under the new condition
 - **overfitting**
 - collect more data for training
- **Details to be discussed in later chapters.**

TP, TN, FP and FN

Model Evaluation



- TP and TN are correct predictions. FP and FN are wrong predictions.

Example - Email Spam detection

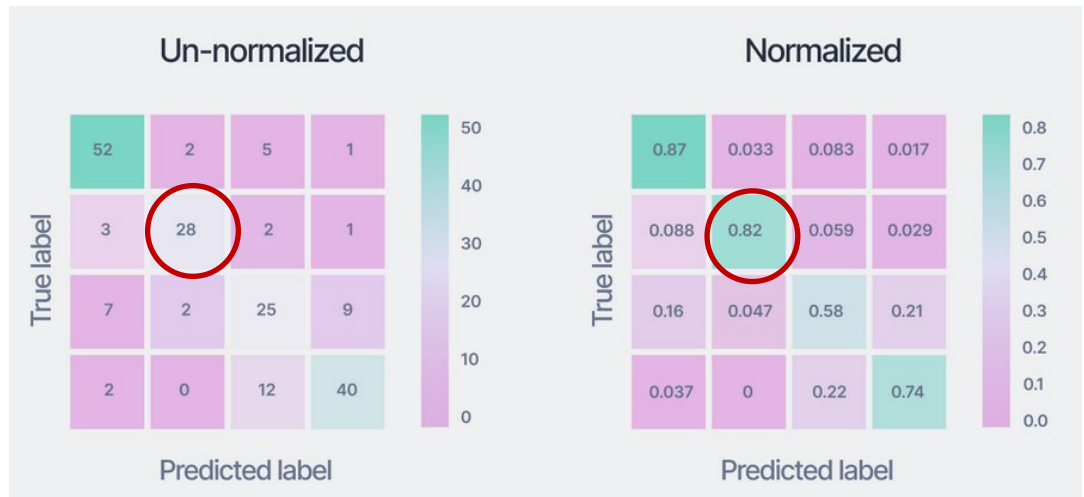
Model Evaluation



Confusion Matrix

Model Evaluation

		Predicted Label			
		A	B	C	D
True Label	A	52	2	5	1
	B	3	28	2	1
	C	7	2	25	9
	D	2	0	12	40



Source: v7labs.com

- Matrix illustrating the **performance** of a supervised learning NN model for classification problem.
- Utilise concepts of TP, TN, FP and FN.
- A visualization of **error rate** between different class prediction. Number of **correct predictions** is seen in the **diagonal** sloping down from left to right in the matrix.
- **Example**. The '52' in the green box on the upper left-hand corner indicates that there are 52 predictions of label A that is correct (TP).
- **Normalisation** should be applied as dataset may be **imbalanced**, i.e. number of samples for each label may be different.

Accuracy

Model Evaluation

- Number of correct predictions over total predictions.
- Formula: $(TP+TN)/(\text{Total number of predictions})$
- Meaning: **How often is the model correct with respect to all predictions?**
- Can be misleading if classes are imbalance e.g. Detecting a rare disease
- Imbalance implies samples available for each class differ by a huge number.

Precision

Model Evaluation

- Number of positive predictions that are correct
- Formula: $TP/(TP+FP)$
- Meaning: **When the model predicts a positive, how often is it correct?**
- Applicable in YouTube Recommendations. If YouTube recommends a video (Positive prediction), it better be one that viewers like. High precision means fewer annoying recommendations (Low False Positives).

Accuracy and Precision

Model Evaluation

- **Accuracy**. Number of correct predictions over total predictions.
- Formula: $(TP+TN)/(\text{Total number of predictions})$
- Meaning: **How often is the model correct with respect to all predictions?**
- Can be misleading if classes are imbalance e.g. Detecting a rare disease
- Imbalance implies samples available for each class differ by a huge number.
- **Precision**. Number of positive predictions that are correct
- Formula: $TP/(TP+FP)$
- Meaning: **When the model predicts a positive, how often is it correct?**
- Applicable in YouTube Recommendations. If YouTube recommends a video (Positive prediction), it better be one viewers like. High precision means fewer annoying recommendations (Low False Positives).

Recall/Sensitivity

Model Evaluation

- Number of positive predictions over total number of positive cases.
- Formula: $TP / (TP + FN)$
- Meaning: **Out of all the actual positive cases, how many did we manage to find?**
- Applicable in **Cancer Detection**. We must find every actual case. We cannot afford to miss one (False Negative). We prefer Recall over Precision here; a false alarm (FP) is better than a missed diagnosis (FN).

F1

Model Evaluation

- **F1**. Harmonic Mean of **Precision** and **Recall**. Provides a **single metric** that weights **both ratios** (precision and recall) in a **balanced** way, requiring both to have high value for the F1-score value to rise.
- Formula: $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$
- Meaning: **The harmonic mean of Precision and Recall.**
- Applicable when you need a balance between Precision and Recall, or when your dataset is uneven (imbalanced classes). It penalizes extreme values.
- For example, if **Precision = 0.01** and **Recall = 1.0**.
=> **Arithmetic mean** = $(0.01 + 1.0) / 2 = 0.505$.
=> **F1** = $2 * (0.01 * 1.0) / (0.01 + 1.0) = \sim 0.02$.

END